

Sincronização de relógios lógicos

STD29006 – Engenharia de Telecomunicações

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Banco de dados transacional I



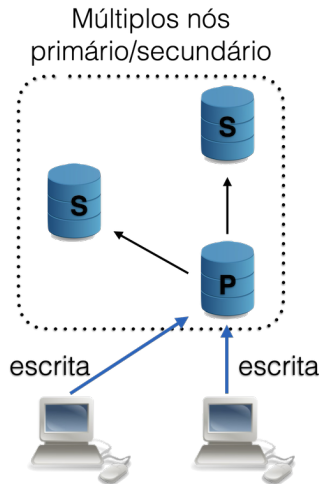
- **Banco de dados transacional** garante a integridade dos dados e a consistência das operações
 - ACID (Atomicidade, Consistência, Isolamento e Durabilidade)
- Pedidos concorrentes de leitura não afetam a consistência
- Pedidos concorrentes de escrita podem ser resolvidos com filas

Banco de dados transacional II

- Em alguns casos é desejado ter múltiplas réplicas do banco de dados
 - Alta disponibilidade, desempenho (réplica mais próxima do cliente)
- Como garantir que todas as réplicas estejam sempre consistentes?
 - **Replicação primário-secundário**
 - **Replicação primário-primário**

Banco de dados distribuído

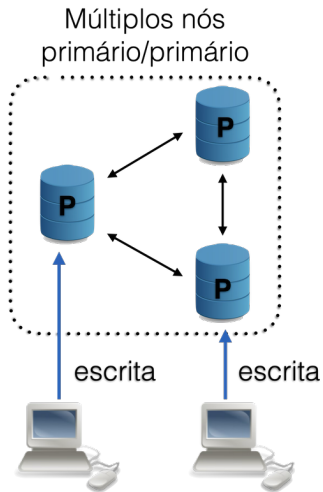
Replicação primário-secundário



- Apenas uma réplica (primário) aceita gravações, as outras (secundárias) são usadas apenas para leitura
- Para garantir a consistência, as gravações são propagadas do primário para os secundários
- Isso melhora a disponibilidade e a escalabilidade, mas pode causar latência na propagação das gravações
- As réplicas secundárias **aplicam as gravações na mesma ordem** em que foram feitas no primário

Banco de dados distribuído

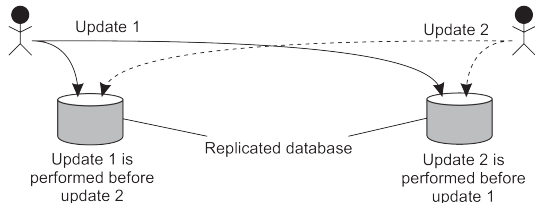
Replicação primário-primário



- Cada réplica é considerada um primário, e as gravações podem ser feitas em qualquer uma delas
- Isso melhora a disponibilidade e a escalabilidade, mas traz desafios de consistência
- Para garantir a consistência, é **necessário um mecanismo de sincronização** entre as réplicas

Banco de dados distribuído

Replicação primário-primário: como garantir a consistência?



- Cliente deposita R\$ 100,00 (saldo inicial: R\$ 1.000,00)
 - Saldo na réplica 1: R\$ 1.111,00
- Ao mesmo tempo, gerente aplica 1% de juros
 - Saldo na réplica 2: R\$ 1.110,00

- Se os relógios físicos das réplicas estiverem sincronizados e cada mensagem for carimbada com uma marcação temporal
 - Isso seria suficiente para garantir a consistência?

Banco de dados distribuído

Por que não basta sincronizar relógios físicos?

- **Relógios físicos** em computadores diferentes nunca estarão perfeitamente sincronizados devido a atrasos de rede, *drift* e falhas
 - Soluções como NTP reduzem, mas não eliminam, a diferença
- Pequenas diferenças podem causar grandes problemas em sistemas distribuídos (ex: ordens de transações, logs).

Relógios Lógicos

Relógio lógico

- **Relógios físicos** são usados para medir o tempo
- **Relógios lógicos** são usados para garantir a ordem dos eventos em um sistema distribuído, **independentemente do tempo real**

Relógio Lógico de Lamport (1978)

O que é um Relógio Lógico de Lamport?

- É um **contador** implementado em software, utilizado para atribuir uma marcação temporal (*timestamp*) a cada evento em um sistema distribuído
- Permite determinar a ordem relativa dos eventos, mesmo sem sincronização de relógios físicos
- Fundamental para garantir a consistência em sistemas distribuídos

Premissas do Relógio Lógico de Lamport

■ Independência

- Se dois processos não interagem, não é necessário sincronizar seus relógios lógicos

■ Comunicação

- Quando processos trocam mensagens, é possível estabelecer uma relação de precedência entre eventos

■ Objetivo

- Garantir que todos os processos concordem sobre a ordem dos eventos que são potencialmente relacionados

Relação de Causalidade

■ Causalidade (causa e efeito)

- A relação $a \rightarrow b$ indica que o evento a ocorreu antes do evento b e pode ter influenciado b
- Essa relação define uma **ordem parcial** entre os eventos do sistema

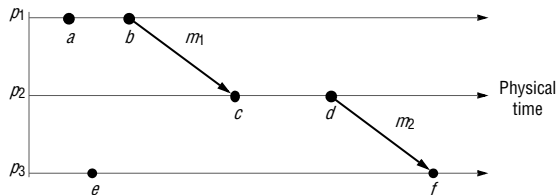
■ Eventos concorrentes

- Se não existe relação causal entre dois eventos, eles são considerados **concorrentes** ($a \parallel b$)

Nota

O relógio lógico de Lamport permite identificar relações de precedência, mas não distingue eventos concorrentes

Relógio Lógico de Lamport: causalidade e concorrência



■ Causalidade

- $a \rightarrow b$ se **a** e **b** são eventos em um mesmo processo e **a** ocorre antes de **b**
- Se um processo P_i envia uma mensagem m para um processo P_j , o evento **send(m)** acontece antes do **receive(m)**: $b \rightarrow c$
- Por transitividade, se $a \rightarrow b$ e $b \rightarrow c$, então $a \rightarrow c$

■ Concorrência

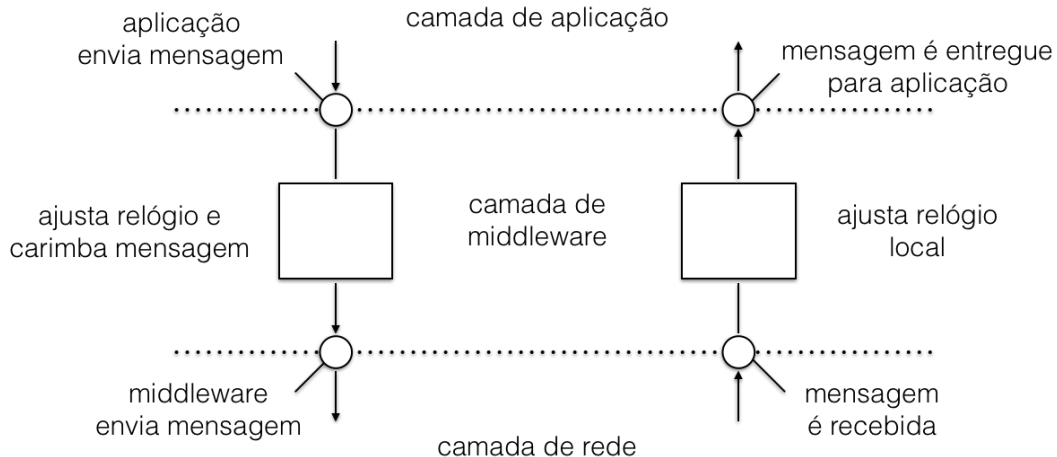
- $a \parallel e$, se **a** e **e** são eventos em processos distintos e não há relação causal entre eles

Relógio Lógico de Lamport: regras

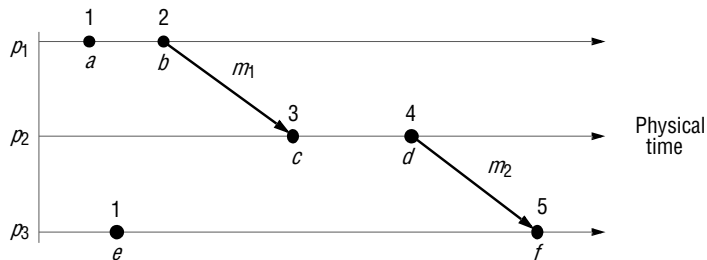
Cada processo P_i mantém seu próprio relógio lógico L_i , que é usado para atribuir uma marcação temporal (*timestamp*) a cada evento

- 1 Antes de cada **evento local** (ação interna ou envio de mensagem), o processo P_i incrementa L_i em 1
- 2 Ao **enviar uma mensagem**, P_i anexa o valor atual de L_i como *timestamp* da mensagem: $t = L_i$
- 3 Ao **receber uma mensagem** (m, t) , o processo P_j atualiza seu relógio: $L_j = \max(L_j, t) + 1$ antes de processar a mensagem

Relógio Lógico de Lamport (1978)



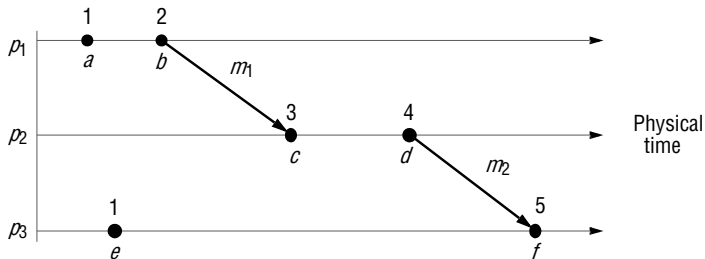
Relógio Lógico de Lamport: exemplo



- L_1, L_2 e L_3 começam com 0
- P_1 anexa $t = L_1 = 2$ a mensagem m_1 antes de enviar para P_2
- P_2 ao receber (m_1, t) faz $L_2 = \max(0, 2) + 1$, logo $L_2 = 3$

Relógio Lógico de Lamport: limitação

Não permite distinguir eventos concorrentes, apenas garante a ordem causal quando ela existe



- Se $e \rightarrow e'$, então necessariamente $L(e) < L(e')$
 - Exemplo: $a \rightarrow c$
- No entanto, $L(e) < L(e')$ não garante que $e \rightarrow e'$
 - Exemplo: $L(e) < L(b)$, mas e e b são concorrentes ($e \parallel b$)

Relógio Lógico de Lamport: ordem causal

■ Ordem causal (parcial)

- Dois eventos a e b têm relação causal ($a \rightarrow b$) se a pode influenciar b direta ou indiretamente
- O relógio de Lamport garante que, se $a \rightarrow b$, então $L(a) < L(b)$
- Porém, o inverso não é verdadeiro: $L(a) < L(b)$ não implica necessariamente $a \rightarrow b$
- Eventos sem relação causal são chamados de **concorrentes** ($a \parallel b$)

■ Ordem parcial

- A relação de causalidade define apenas uma ordem parcial entre os eventos do sistema distribuído
- Nem todos os eventos podem ser comparados: alguns são concorrentes

Relógio Lógico de Lamport: ordem total

■ Ordem total

- Para garantir que todos os eventos possam ser comparados, mesmo os concorrentes
- Cada evento é identificado por um par (T, i) , onde T é o relógio de Lamport e i é o identificador do processo
- A comparação é feita assim:
 - $(T_i, i) < (T_j, j)$ se $T_i < T_j$, ou
 - $T_i = T_j$ e $i < j$
- Assim, mesmo eventos concorrentes recebem uma ordem arbitrária, mas consistente entre todos os processos

■ Aplicação

- Essencial para algoritmos que exigem desempate, como ordenação de mensagens em sistemas distribuídos

Difusão seletiva (*multicast*) com ordem total

- Em sistemas distribuídos, é comum que múltiplos processos precisem receber e processar mensagens na **mesma ordem**, para garantir consistência entre réplicas
- **Difusão seletiva com ordem total** (*total order multicast*) garante que todas as réplicas processem as operações na mesma sequência, mesmo com mensagens concorrentes
- Fundamental em bancos de dados distribuídos, sistemas de mensagens e coordenação distribuída

Desafios da ordem total

- **Concorrência:** Mensagens podem ser enviadas simultaneamente por diferentes processos
- **Atrasos de rede:** Mensagens podem chegar em ordens diferentes para cada processo
- **Necessidade:** Todos os processos devem concordar sobre a ordem de entrega das mensagens, independentemente da ordem de chegada

Exemplo prático

Dois clientes fazem depósitos simultâneos em réplicas diferentes de um banco. A ordem dos depósitos deve ser a mesma em todas as réplicas para manter a consistência

Premissas para difusão seletiva com ordem total

- **Sem perda de mensagens:** Todas as mensagens enviadas são eventualmente entregues
- **Ordem FIFO por emissor:** Mensagens enviadas por um mesmo processo são recebidas na ordem de envio
- **Entrega para o próprio emissor:** O processo que envia uma mensagem também a recebe

Algoritmo de difusão seletiva com ordem total (Lamport)

- 1 Quando um processo P_i deseja enviar uma mensagem m_i , ele anexa seu **relógio de Lamport** e envia para todos os processos (incluindo ele mesmo)
- 2 Cada processo, ao receber uma mensagem, armazena-a em uma **fila ordenada** pelo *timestamp* e envia uma confirmação (**ACK**) para todos os outros processos
- 3 Uma mensagem m_i só pode ser entregue à aplicação quando:
 - m_i está no início da fila (menor *timestamp* entre todas as mensagens na fila)
 - m_i foi confirmada (ACK) por todos os processos

Por que usar relógio lógico de Lamport?

- O relógio lógico de Lamport fornece uma ordem parcial consistente com a causalidade: se $a \rightarrow b$, então $L(a) < L(b)$
- Para garantir uma ordem total (mesmo entre eventos concorrentes), desempata-se usando o identificador do processo: (T, i)
- Assim, todos os processos concordam sobre uma ordem total arbitrária, mas consistente, mesmo que a ordem real de chegada das mensagens varie

Importante

A ordem total obtida é determinística e igual para todos os processos, mas não reflete necessariamente a causalidade real entre eventos concorrentes

Difusão seletiva com ordem total

Limitações

- Pode introduzir latência (espera por ACKs)
- A ordem total pode ser arbitrária para eventos concorrentes, pois o relógio de Lamport não distingue causalidade de concorrência
- Para identificar eventos realmente concorrentes, são necessários **relógios vetoriais**

Relógios vetoriais

O que é um Relógio Vetorial?

- **Relógio vetorial** é uma estrutura de dados usada em sistemas distribuídos para **rastrear a relação causal entre eventos**
- Permite identificar não apenas a ordem, mas também se dois eventos são concorrentes ou causais

Relógio Vetorial vs. Relógio de Lamport

■ Relógio de Lamport

- Garante: $a \rightarrow b \Rightarrow L(a) < L(b)$
- Não garante o inverso: $L(a) < L(b)$ não implica $a \rightarrow b$
- Não distingue eventos concorrentes

■ Relógio Vetorial

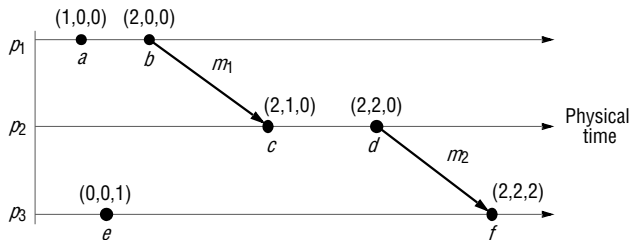
- Garante: $a \rightarrow b \Leftrightarrow V(a) < V(b)$ (comparação elemento a elemento)
- Permite identificar eventos concorrentes: $V(a) \not< V(b)$ e $V(b) \not< V(a)$, ou seja, $a \parallel b$
- Fornece uma visão mais precisa da causalidade

Relógio vetorial: regras para atualização

- Em um sistema com N processos, cada processo P_i mantém um vetor V_i de tamanho N
 - $V_i[j] = 0$ para todo j
- **Evento local em P_i**
 - $V_i[i] \leftarrow V_i[i] + 1$
- **Envio de mensagem**
 - P_i envia uma cópia de V_i junto com a mensagem
- **Recebimento de mensagem em P_j**
 - Para cada k : $V_j[k] \leftarrow \max(V_j[k], V_{msg}[k]), j \neq k$
 - $V_j[j] \leftarrow V_j[j] + 1$
 - Exemplo:
 - $j = 4, V_j = \{4, 9, 11, 3\}$ e foi recebido $\{2, 5, 15, 3\}$
 - Resultado: $V_j = \{4, 9, 15, 4\}$

Relógio vetorial

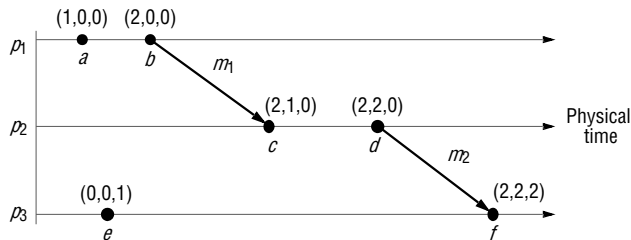
Exemplo



- **Precedência causal:** $b \rightarrow d$ se, e somente se, $V(b) < V(d)$ (comparação elemento a elemento)
 - Exemplo: $V(b) = \{2, 0, 0\}$ e $V(d) = \{2, 2, 0\}$
 - Comparando: $2 \leq 2 \wedge 0 \leq 2 \wedge 0 \leq 0 \Rightarrow$ todos verdadeiros
 - Portanto, $b \rightarrow d$ (relação causal)

Relógio vetorial

Exemplo



■ **Sem relação causal:** $f \rightarrow c$? Verifique se $V(f) < V(c)$

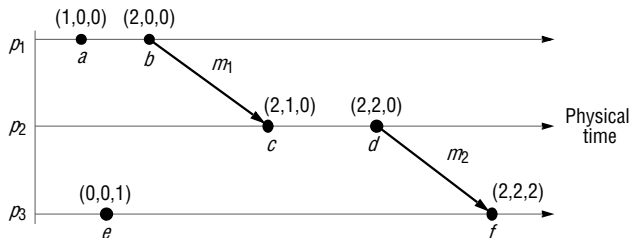
■ $V(f) = \{2, 2, 2\}$, $V(c) = \{2, 1, 0\}$

■ Comparando: $2 \leq 2$ (V), $2 \leq 1$ (F), $2 \leq 0$ (F)

■ Como nem todos são verdadeiros, f não precede c

Relógio vetorial

Exemplo



■ **Concorrência:** $a \parallel e$ se $V(a) \not\leq V(e)$ e $V(e) \not\leq V(a)$

■ $V(a) = \{1, 0, 0\}$, $V(e) = \{0, 0, 1\}$

■ $V(a) < V(e)$? $1 \leq 0 \wedge 0 \leq 0 \wedge 0 \leq 1 \Rightarrow$ Falso

■ $V(e) < V(a)$? $0 \leq 1 \wedge 0 \leq 0 \wedge 1 \leq 0 \Rightarrow$ Falso

■ Portanto, a e e são concorrentes ($a \parallel e$)

Relógios vetoriais

■ Vantagens

- Permite identificar relações causais e concorrência de forma precisa
- Garante que todos os processos concordem sobre a ordem dos eventos

■ Desvantagens

- Cresce com o número de processos: cada processo precisa manter um vetor do tamanho do número de processos
- Pode ser mais complexo de implementar e gerenciar do que o relógio lógico de Lamport

■ Aplicações

- Sistemas de mensagens distribuídas (ex: Kafka, RabbitMQ)
- Bancos de dados distribuídos (ex: Cassandra, Amazon DynamoDB)
- Sistemas de arquivos distribuídos (ex: Google File System, HDFS)

Curiosidades

Como a Amazon AWS elege um líder

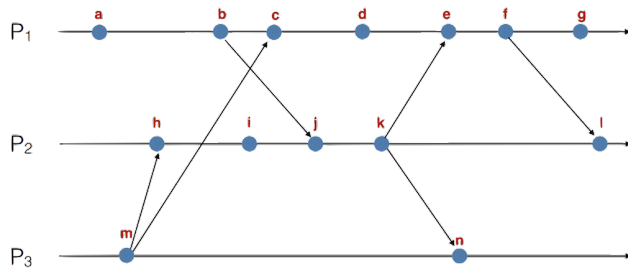
- Eleição de líder em sistemas distribuídos é um problema clássico
- **Amazon AWS** usa um algoritmo de eleição de líder baseado em relógios lógicos¹

"Evitamos depender do tempo em sistemas distribuídos... É difícil garantir que os relógios do sistema em um cluster estejam sincronizados o suficiente para depender dessa sincronização para ordenar ou coordenar operações distribuídas"

¹ <https://aws.amazon.com/pt/builders-library/leader-election-in-distributed-systems/>

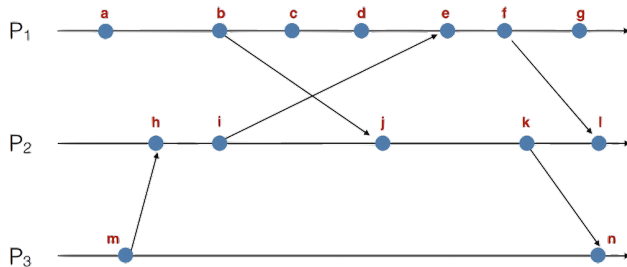
Exercícios

Exercício 1: Relógio lógico de Lamport



- 1 Coloque o valor do relógio em cada evento
- 2 Ciente que o evento **i** ocorreu antes de **n**, o que pode-se afirmar sobre seus relógios lógicos?
- 3 Sabe-se que $L(h) < L(c)$, logo é possível afirmar a relação de precedência entre **h** e **c**?

Exercício 2: Relógio vetorial



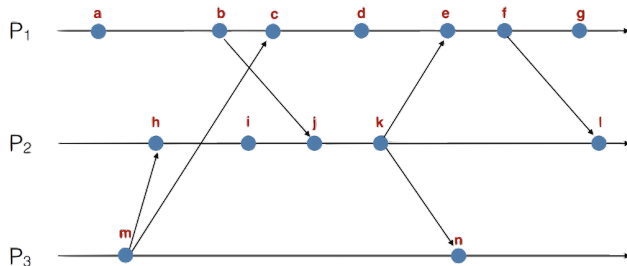
- 1 Coloque o valor do relógio vetorial em cada evento
- 2 Comprove a relação $a \rightarrow j$ comparando os relógios destes eventos
- 3 Compare os relógios de **d** e **j** e comprove a relação $d \parallel j$

Exercício 3: Relógio vetorial

Considere que, em um sistema distribuído com vários processos, ao final de sua execução o processo P_2 possui o seguinte relógio vetorial: $\{3, 1, 0, 6, 4\}$.

- 1 Quantos eventos ocorreram localmente em P_2 ?
- 2 Quantos eventos do processo P_3 já foram observados por P_2 ? Esse valor corresponde ao total de eventos que realmente ocorreram em P_3 ?
- 3 Quantos processos existem no sistema distribuído?
- 4 Quantas mensagens P_2 recebeu de outros processos ao longo da execução?

Exercício 4: Relógio lógico de Lamport



- 1 No algoritmo original, dois eventos em processos diferentes podem ter o mesmo *timestamp*.
 - Pesquise como Lamport propôs garantir *timestamps* únicos (dica: uso do identificador do processo).
 - Preencha os valores dos relógios em cada evento, considerando *timestamps* únicos.

Aulas baseadas em



TANENBAUM, Andrew S; STEEN, Maarten van. **Sistemas Distribuidos: Princípios e paradigmas**. 2. ed.: Prentice Hall, 2008.